Program, Process, and Thread in Java

Before learning multithreading, it is important to understand the relationship between:

```
Program
Process
Thread
CPU Core
Concurrency
Parallelism
```

These concepts describe different stages of how code moves from a stored file to actual execution.

---

1. What Is a Program?

A **program** is a set of instructions written to make a computer perform a particular task.

Examples include:

- A Java source file
- A compiled Java class file
- A browser application
- A code editor
- A media player

Java example:

```
public class Demo {

```

```
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

This source code can be stored in:

```
Demo.java
```

After compilation, Java bytecode is stored in:

```
Demo.class
```

Both are programs in stored form.

They contain instructions, but those instructions are not currently being executed.

Program as a Passive Entity

A program is considered a **passive entity**.

```
Program
→ instructions stored in a file
→ not currently executing
```

Examples:

```
Chrome executable stored on disk
VS Code application stored on disk
Demo.class stored on disk
```

The program file itself occupies storage space.

It does not receive its own active process memory, CPU scheduling, or execution resources until it is launched.

Operating systems may cache file contents in memory for performance, but that does not mean the program is actively executing as a process.

Is Bytecode Also a Program?

Yes.

Java bytecode stored inside a `.class` file is still a set of instructions.

```
Java source code
→ human-readable program

Java bytecode
→ JVM-executable program representation
```

Bytecode becomes active only when a JVM loads and executes it.

---

2. What Is a Process?

A **process** is a running instance of a program.

```
Program
→ passive instructions

Process
→ active execution of those instructions
```

When we run:

```
java Demo
```

the operating system starts a process that contains the Java Virtual Machine and the Java application being executed.

Real-Life Examples

When we launch applications such as:

- Chrome
- VS Code
- Spotify
- IntelliJ IDEA
- A Java application

the operating system creates one or more processes.

A modern application may use several processes internally.

For example, a browser may create separate processes for:

- Browser interface
- Individual tabs
- Extensions
- Graphics processing
- Network services

Therefore, one application window does not always correspond to exactly one process.

What Happens When a Process Starts?

When a process is created, the operating system provides it with an execution environment.

This includes:

3. Memory

The process receives a virtual address space that can contain:

- Executable code
- Runtime data
- Heap memory
- Thread stacks

- Loaded libraries
- Memory-mapped files

---

4. CPU Scheduling

The process contains one or more threads.

The operating system scheduler decides when those threads receive processor time.

The process itself does not directly execute instructions. Its threads do.

---

5. Resources

A process may use:

- Files
- Network sockets
- Input and output devices
- Database connections
- Shared libraries
- Operating-system handles

Important Properties of a Process

6. Independent Execution Environment

Each process has its own execution context.

If one process crashes, other processes normally continue running.

However, applications can still affect one another indirectly through shared services, files, databases, or operating-system resources.

---

7. Separate Address Space

Processes do not share their normal memory directly by default.

```
Process A memory
≠
Process B memory
```

One process cannot normally access another process's memory directly.

Communication between processes requires specific mechanisms such as:

- Pipes
- Sockets
- Shared memory
- Files
- Message queues
- Remote calls

This is called **inter-process communication**.

---

8. Managed by the Operating System

The operating system manages:

- Process creation
- Process termination
- Memory allocation
- CPU scheduling
- Resource access
- Security boundaries

Where Does the JVM Fit?

When we run:

```
java Demo
```

the Java launcher starts a process containing a JVM.

A more accurate mental model is:

```
Operating-system process
└── JVM runtime
    └── Java application
```

The JVM is not a separate process living inside another unrelated Java process.

It is the runtime executing within the operating-system process started for the Java application.

---

9. What Is a Thread?

A **thread** is an independent path of execution inside a process.

A process can contain one or more threads.

```
Program
→ launched as a process
→ process contains threads
```

A thread executes instructions belonging to the process.

Thread as an Execution Unit

A thread has its own execution-related state, including:

- Program counter
- Call stack
- Stack frames
- Local variables
- Current instruction position

- Native execution state

Threads inside the same process generally share:

- Heap objects
- Static variables
- Loaded classes
- Files opened by the process
- Network resources

This gives us the core multithreading model:

```
Private execution stacks
+
shared process memory
```

Is a Thread a Lightweight Process?

A thread is often called a **lightweight process** because:

- Threads are cheaper to create than processes
- Threads share the same process memory
- Communication between threads is faster
- Switching between threads can be cheaper than switching between processes

However, a thread is not a separate process.

A thread does not normally have its own independent address space.

Process and Thread Relationship

```
One process
├── Main thread
├── Worker thread 1
```

```
├── Worker thread 2
└── Background thread
```

A process must contain at least one executing thread.

When a normal Java application begins, the JVM creates a thread that invokes the application's `main()` method.

This thread is commonly called the:

```
main thread
```

How Java Code Runs

Consider:

```java
public class Main {

    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

The execution passes through several stages.

Step 1: Write Source Code

The Java source code is stored in:

```
Main.java
```

It contains human-readable Java instructions.

Step 2: Compile the Source Code

Compile using:

```
javac Main.java
```

This produces:

```
Main.class
```

The `.class` file contains Java bytecode.

```
Java source code
→ javac compiler
→ Java bytecode
```

Bytecode is not tied directly to one processor architecture.

It is designed to be executed by a compatible JVM.

Step 3: Launch the Application

Run:

```
java Main
```

The Java launcher asks the operating system to start a process.

That process initializes a JVM runtime.

Step 4: JVM Initialization

The JVM performs tasks such as:

- Loading required classes
- Verifying bytecode

- Linking classes
- Initializing runtime services
- Preparing runtime memory areas
- Starting garbage-collection support
- Preparing the main execution thread
- Invoking `main()`

Step 5: Class Loading

The JVM loads:

```
Main.class
```

The class-loading process broadly includes:

```
Loading
→ Linking
→ Initialization
```

Linking itself contains:

```
Verification
Preparation
Resolution
```

For beginner-level understanding:

> The JVM loads the class, checks that the bytecode is valid, prepares class data, and makes the class ready for execution.

Step 6: Runtime Memory Areas

The JVM organizes memory into several runtime areas.

A simplified view is:

```
Java Process
└── JVM Runtime
    ├── Heap
    ├── Method Area / Metaspace implementation
    ├── Java Stacks
    ├── PC Registers
    └── Native Method Stacks
```

Heap

The heap stores objects and arrays.

```java
Student student = new Student();
```

The `Student` object is normally created in heap memory.

The heap is shared among threads in the same JVM.

Method Area

The Java Virtual Machine Specification defines a logical area called the **method area**.

It stores information such as:

- Class metadata
- Method metadata
- Runtime constant pools
- Static field-related class data
- Method bytecode

In HotSpot JVM implementations, class metadata is commonly stored in:

```
Metaspace
```

The method area is a JVM-level concept. Metaspace is one implementation approach used by HotSpot.

Java Stack

Each Java thread has its own Java stack.

```
Main thread
→ main-thread stack

Worker thread
→ worker-thread stack
```

A stack contains stack frames for active method calls.

Example:

```
main()
→ methodA()
→ methodB()
```

The stack may look like:

```
Thread Stack
├── methodB() frame
├── methodA() frame
└── main() frame
```

Each frame stores information such as:

- Local variables
- Operand stack
- Method execution data
- Return information

Program Counter Register

Each JVM thread has its own program-counter register.

Conceptually, it tracks the current or next JVM instruction for that thread.

```
Thread-1 PC
→ Thread-1 execution position

Thread-2 PC
→ Thread-2 execution position
```

Each thread needs a separate execution position because different threads may be executing different methods.

Native Method Stack

A thread may call native code through mechanisms such as JNI.

The native method stack supports execution outside ordinary Java bytecode.

JVM Memory vs Operating-System Process Memory

The operating system sees a process with virtual memory regions.

The JVM interprets and manages portions of that process memory according to JVM concepts such as:

- Heap
- Method area
- Java stacks
- Native structures
- JIT-compiled code cache

It is not accurate to say that:

```
OS code segment = JVM method area
```

or:

```
OS data segment = JVM method area
```

These are different abstraction layers.

A better model is:

```
Operating-system process memory
└── contains memory used by the JVM
    ├── Java heap
    ├── class metadata
    ├── thread stacks
    ├── JIT code cache
    ├── native libraries
    └── JVM internal structures
```

The JVM introduces its own memory model because Java is a managed runtime environment.

Step 7: The Main Thread Starts

After the JVM prepares the application, it invokes:

```
public static void main(String[] args)
```

on the main thread.

```
Java process
└── JVM
    └── main thread
        └── main() executes
```

The `main()` method always executes inside a thread.

It does not execute independently of the thread model.

Stack Frame for `main()`

When `main()` starts, a stack frame is created on the main thread's Java stack.

```
Main Thread Stack
└── main() Stack Frame
    ├── args reference
    ├── local variables
    ├── operand stack
    └── method execution information
```

If `main()` calls another method:

```
public static void main(String[] args) {
    calculate();
}
```

another frame is added:

```
Main Thread Stack
├── calculate() frame
└── main() frame
```

When `calculate()` returns, its frame is removed.

Creating Additional Threads

The main thread can create worker threads:

```
public class Main {

    public static void main(String[] args) {
        Thread worker1 = new Thread(
                () -> System.out.println("Worker 1")
```

```
        );

        Thread worker2 = new Thread(
                () -> System.out.println("Worker 2")
        );

        worker1.start();
        worker2.start();
    }
}
```

Now the process contains at least:

```
Main thread
Worker thread 1
Worker thread 2
```

Each thread has its own Java stack and PC register, while all three threads can access shared heap objects.

How the CPU Executes Threads

The CPU executes machine instructions.

Java bytecode may be:

- Interpreted by the JVM
- Compiled into native machine code by the JIT compiler
- Executed as optimized native instructions

The CPU ultimately executes native machine instructions, not Java source code directly.

Single-Core CPU

Suppose the system has:

```
One CPU core
```

Only one runnable thread can execute on that core at one exact instant.

If the process contains:

```
Main thread
Thread-1
Thread-2
```

the scheduler may switch among them:

```
Main
→ Thread-1
→ Main
→ Thread-2
→ Thread-1
```

The switching may happen so quickly that the threads appear to run together.

Context Switching

What Is Context Switching?

A **context switch** occurs when the CPU stops executing one thread and begins executing another.

Conceptually:

```
Running Thread-A
→ save Thread-A execution context
→ select Thread-B
→ restore Thread-B execution context
→ continue Thread-B
```

The saved execution context may include:

- CPU registers
- Instruction pointer
- Stack pointer
- Scheduling information
- Processor state

Important Correction

The operating system does not simply copy all CPU register values into the Java stack frame.

The native thread context is saved in operating-system and runtime-managed structures.

Java stack frames store Java method execution data, but operating-system context switching is a lower-level mechanism.

A safe mental model is:

```
Thread execution state is saved
→ another thread's state is restored
```

without assuming that all native register state is written directly into Java stack frames.

What Happens to the Program Counter?

At the JVM level, each thread has its own conceptual PC register for bytecode execution.

At the processor level, the operating system also manages the native instruction position during a context switch.

When the thread runs again, execution continues from the correct saved location.

Cost of Context Switching

Context switching is not free.

It may involve:

- Saving and restoring state
- Scheduler work
- Cache disruption
- Translation lookaside buffer effects
- Moving execution between processor cores

Too many threads can reduce performance because the system spends more time switching than doing useful work.

Concurrency on a Single-Core CPU

On a single-core CPU:

```
Only one thread executes at an exact instant.
```

However, several tasks can make progress during the same time period.

Example:

```
Main executes briefly
→ Worker-1 executes
→ Main executes again
→ Worker-2 executes
```

This is **concurrency**.

Definition of Concurrency

> Concurrency means multiple tasks are in progress during overlapping periods of time, but they do not necessarily execute at the exact same instant.

Concurrency is about:

```
Managing multiple tasks
```

It can exist even with one processor core.

Multi-Core CPU

Suppose the processor has:

```
Four CPU cores
```

Different threads may execute at the same instant:

```
Core-1 → Main thread
Core-2 → Worker-1
Core-3 → Worker-2
Core-4 → Worker-3
```

This is **parallel execution**.

Definition of Parallelism

> Parallelism means multiple tasks are executing at the same physical instant on different processing units.

Parallelism is about:

```
Executing multiple tasks simultaneously
```

It normally requires multiple cores, processors, or hardware execution units.

Concurrency vs Parallelism

Concurrency	Parallelism
Multiple tasks make progress in overlapping time periods	Multiple tasks execute at the same instant
Can happen on one core	Usually requires multiple cores
Concerned with task coordination	Concerned with simultaneous execution
Achieved through scheduling and interleaving	Achieved through actual hardware execution
Does not guarantee speed improvement	Can improve throughput when work is parallelizable

A program can be:

```
Concurrent but not parallel
```

Example:

```
Several threads on one CPU core
```

A program can also be:

```
Concurrent and parallel
```

Example:

```
Several coordinated threads executing on multiple cores
```

Multitasking

What Is Multitasking?

Multitasking means the operating system manages multiple tasks or applications during the same time period.

Examples:

```
Browser running
+
code editor running
+
music player running
```

These applications normally run in separate processes.

The operating system schedules their threads across available CPU cores.

Multithreading

What Is Multithreading?

Multithreading means one process contains multiple threads of execution.

```
One Java process
├── Main thread
├── Request-processing thread
├── Database worker thread
└── Background monitoring thread
```

These threads share the process's resources and memory.

Multitasking vs Multithreading

Multitasking	Multithreading
Usually involves multiple processes or applications	Involves multiple threads inside one process
Processes have separate address spaces	Threads share the process address space
Communication is relatively heavier	Communication through shared memory is easier
Stronger isolation	Lower isolation

Multitasking	Multithreading
Process creation is usually more expensive	Thread creation is usually cheaper
One process crash usually does not directly terminate unrelated processes	A serious failure can affect the entire process

Process vs Thread

Process	Thread
Running instance of a program	Execution path inside a process
Has an independent address space	Shares process address space
Contains one or more threads	Belongs to one process
More expensive to create	Usually cheaper to create
Communication requires IPC mechanisms	Threads communicate through shared memory
Better isolation	Easier communication but more race-condition risk
Context switching is generally heavier	Context switching is generally lighter

Example: Java Web Application

Consider a Spring Boot application.

```
Spring Boot JAR
→ program stored on disk
```

When started:

```
java -jar application.jar
→ operating-system process
```

Inside that process:

```
JVM
├── main thread
├── garbage-collection threads
├── web-server threads
├── database-pool support threads
└── application worker threads
```

Several request-processing threads may run the same controller code for different HTTP requests.

These threads share application objects stored in the heap.

This is why thread safety matters in server applications.

Example: Browser

A browser may use multiple processes:

```
Browser process
Renderer process
GPU process
Extension process
Network process
```

Each process may also contain multiple threads.

Therefore:

```
Application
→ may contain multiple processes
→ each process may contain multiple threads
```

Complete Execution Flow of a Java Program

```
1. Write Main.java
2. Compile with javac
3. Generate Main.class bytecode
```

```
4. Run using java Main
5. Operating system starts a process
6. JVM initializes inside that process
7. JVM loads and verifies classes
8. JVM prepares runtime memory
9. JVM invokes main() on the main thread
10. Main thread executes Java code
11. Additional threads may be created
12. Scheduler assigns runnable threads to CPU cores
13. JVM interprets or JIT-compiles bytecode
14. CPU executes native machine instructions
15. Process ends when its non-daemon work is complete or it is terminated
```


Important Rules

1. A program is stored instructions; a process is an executing instance.
2. A `.class` file contains bytecode and is not itself an executing process.
3. Running `java Main` starts an operating-system process containing a JVM.
4. A process has its own virtual address space and resources.
5. Threads are execution paths inside a process.
6. Threads share heap data but have separate stacks.
7. The `main()` method executes on the main thread.
8. Each JVM thread has its own Java stack and PC register.
9. The CPU ultimately executes native machine instructions.
10. Single-core systems can provide concurrency through interleaving.
11. Multi-core systems can provide true parallel execution.
12. Context switching saves and restores native thread execution state.
13. Native register state is not simply copied into Java stack frames.
14. Concurrency means overlapping progress; parallelism means simultaneous execution.
15. Multitasking commonly involves multiple processes.
16. Multithreading involves multiple threads inside one process.

17. Threads are cheaper than processes but have weaker isolation.
18. Shared memory makes thread communication fast but introduces race-condition risks.
19. JVM memory areas and operating-system memory segments are different abstraction layers.
20. Too many threads can reduce performance because of scheduling and context-switching overhead.

Final Summary

```
Program
→ stored instructions

Process
→ running instance of a program

Thread
→ independent path of execution inside a process
```

```
Java source code
→ javac
→ bytecode
→ java launcher
→ operating-system process
→ JVM initialization
→ main thread
→ main() execution
```

```
Single core
→ concurrent interleaving

Multiple cores
→ possible parallel execution
```

```
Multitasking
→ multiple processes or applications
```

```
Multithreading
→ multiple threads inside one process
```

The central idea is:

> A program becomes active as a process, and the actual work inside that process is performed by threads.